UNIVERSITY OF WESTMINSTER

INFORMATICS INSTITUTE OF TECHNOLOGY

University of Westminster

4COSC001W: Software Development I Coursework 2026 April

NAME	MOHAMED QWAMY MOHAMED QUMAIR
IIT NO	20251201
UOW NO	22125754

Main code flowchart: click here to see the flow chart

PSEUDOCODE

1. Pseudocode for validate_recipe_name()

```
FUNCTION validate_recipe_name
    LOOP forever
        INPUT recipe_name
        IF recipe_name is empty THEN
            PRINT "Recipe name can't be empty"
            CONTINUE loop
        ENDIF

        IF length of recipe_name < 3 OR length > 50 THEN
            PRINT "Recipe name should contain between 3 and 50 characters"
            CONTINUE loop
        ENDIF

        IF recipe_name contains only spaces THEN
            PRINT "Recipe name can't contain only spaces"
            CONTINUE loop
        ENDIF
        valid = TRUE
        letter_found = FALSE
        FOR each character in recipe_name

            IF character is a letter THEN
                letter_found = TRUE
            ELSE IF character is space OR '-' OR "'" THEN
                CONTINUE to next character
            ELSE
                PRINT "Only letters, hyphen (-), apostrophe ('), and space are allowed"
                SET = FALSE
                BREAK loop
            ENDIF
        ENDFOR
        IF valid is FALSE THEN
            CONTINUE loop
        ENDIF

        IF letter_found is FALSE THEN
            PRINT "Recipe name must contain at least one letter"
            CONTINUE loop
        ENDIF
        RETURN recipe_name
    ENDLOOP
```

```
END FUNCTION
```

2. Pseudocode for validate_ingredient_name()

```
FUNCTION validate_ingredient_name
    CREATE empty list main_ing
    CREATE empty list ing

    LOOP forever
    TRY
        INPUT ingr_count
        IF ingr_count is between 3 and 20 THEN
            BREAK loop
        ELSE
            PRINT "Number of ingredients must be between 3 and 20"
        ENDIF

    CATCH error (invalid number)
        PRINT "Please provide the ingredient count as a whole number"
    ENDTRY
    ENDLOOP

    SET count = 0
    WHILE count < ingr_count
        LOOP forever
            INPUT ing_name
            IF length of ing_name < 3 OR length > 30 THEN
                PRINT "Ingredient name should be between 3 and 30 characters"
                CONTINUE loop
            ENDIF

            CONVERT ing_name to lowercase
            IF ing_name exists in ing list THEN
                INPUT ask ("Duplicate ingredient detected! Add anyway? (yes/no)")
                IF ask is not "yes" THEN
                    CONTINUE loop
                ENDIF
            ENDIF
            ADD ing_name to ing list
            CALL validate_ingredient_details(ing_name)
            STORE result in ing_quan_unit
            ADD ing_quan_unit to main_ing
            INCREMENT count by 1
            BREAK loop
        ENDLOOP
    ENDWHILE
```

```
        RETURN main_ing
END FUNCTION
```

3. Pseudocode for validate_ingredient_details(ing_name)

```
FUNCTION validate_ingredient_details(ing_name)
  LOOP forever
    TRY
      INPUT ingredient_quantity as FLOAT
      IF ingredient_quantity is greater than zero THEN
        BREAK loop
      ELSE
        DISPLAY 'Quantity must be greater than zero'
      ENDIF
    CATCH ValueError
      DISPLAY "Enter a valid number"
    ENDTRY
  ENDLOOP
  LOOP forever
    INPUT ingredient_unit
    change ingredient_unit into lowercase
    IF ingredient_unit in units THEN
      BREAK loop
    ELSE
      DISPLAY "invalid unit !"
    ENDIF
  ENDLOOP
  RETURN ing_name,ingredient_quantity,ingredient_unit
ENDFUNCTION
```

1. Pseudocode for recipe_time_validation

```
FUNCTION recipe_time_validation
  FOREVER Loop
    TRY
      INPUT time
      IF length of time is 5 AND subindex 2 of time is ":" THEN
        Hour SET subindex from 0 to 2 index as integer
        Min SET subindex from 3 to last index as integer
        IF Hour not in range of 0 and 12 AND Min not in 0 and 59 THEN
          DISPLAY "Invalid time range,Time must be between 00:05 and 12:00"
          CONTINUE Loop
        ENDIF
        IF (Hour*60+ Min) is in the range of 5 and 720 THEN
          RETURN time
        ELSE
```

```
            DISPLAY ("Time must be between 00:05 and 12:00.")
          ENDIF
        ELSE
          DISPLAY"Format must be HH:MM"
        ENDIF
      CATCH ValueError
        DISPLAY "Invalid format."
      ENDTRY
    ENDLOOP
ENDFUNCTION
```

4. Pseudocode for recipe_category_tags_validation

```
FUNCTION recipe_category_tags_validation
  cate=["BREAKFAST", "LUNCH", "DINNER", "DESSERT", "SNACK", "BEVERAGE"]
  FOREVER Loop
    INPUT rec_cate
    IF rec_cate IN cate THEN
      BREAK Loop
    DISPLAY "Invalid category."
    ENDIF
  ENDLOOP
RETURN rec_cate
```

5. Pseudocode for recipe_tags

```
FUNCTION recipe_tags
  tags=empty set
  FOREVER Loop
    INPUT rec_tags
    ADD rec_tags to the tags
    INPUT Permission
    IF permission IS '-' THEN
      DISPLAY "tags added successfuly"
      BREAK Loop
    ENDIF
    IF permission IS "+" THEN
      CONTINUE Loop
    ELSE
      DISPLAY"enter either + or -"
    ENDIF
  ENDLOOP
  RETURN tags
ENDFUNCTION
```

6. Pseudocode for single_display(REC_ID)

```
FUNCION single_display(REC_ID)
```

```
  TRY
    DISPLAY "========================================="
    DISPLAY "Recipe ID: REC_ID"
    DISPLAY "Name: recipes[REC_ID]['name']"
    DISPLAY "Category: recipes[REC_ID]['Category']"
    DISPLAY "Cooking Time: recipes[REC_ID]['time']"
    DISPLAY "---------------------------------------"
    DISPLAY "Ingredients:"
    FOR Ingredient IN recipes[REC_ID]['Ingredient']
      DISPLAY Ingredient subindex O,1,2
    ENDFOR Loop
    DISPLAY "========================================="
  CATCH
    DISPLAY"enter the valid and available ID"
  ENDTRY
ENDFUNCTION
```

7. Pseudocode for summary_display()

```
FUNCTION summary_display
  DISPLAY "Display All Recipes (Summary)"
  DISPLAY "Total Recipes: (length of recipes)"
  DISPLAY "---------------------------------------"
  FOR each key IN recipes
    DISPLAY "Recipe ID:{key} | {recipes[key]['name']} | {recipes[key]['Category']} | {recipes[key]['time']}"
  FOR Loop
ENDFUNCTION
```

8. Pseudocode for save_rec

```
FUNCTION save_rec
  OPEN file "recipes.txt" in WRITE mode
  TRY
    IF number of recipes > 0 THEN
      FOR each key, value in recipes
        WRITE "===RECIPE===" to file
        WRITE "ID:" + key
        WRITE "NAME:" + recipe name
        WRITE "CATEGORY:" + category
        WRITE "TIME:" + cooking time
        WRITE "Ingredients:"
        FOR each ingredient in Ingredients list
          WRITE ingredient name + "|" + quantity + "|" + unit
        ENDFOR
        CONVERT tags set into comma-separated string
        WRITE "TAGS:" + tags
        WRITE "===END==="
      ENDFOR
```

```
        CLOSE file
        DISPLAY "Recipes saved successfully"
      ELSE
        DISPLAY "You must enter at least one recipe to save"
      ENDIF
    CATCH error
      DISPLAY "Error: Cannot save file"
END FUNCTION
```

9. Pseudocode for prevsa

```
FUNCTION prevsa
  global recipes
  global rec_count
  OPEN "recipes.txt" FOR READ
  SET rec_num = empty string
  LOOP forever
    READ a line from file INTO read
    IF read is empty THEN
      BREAK loop
    ENDIF
    REMOVE spaces from read
    IF read starts with "ID:" THEN
      SET rec_num = substring of read from index 3 to end
      CREATE empty dictionary for recipes[rec_num]
      CREATE empty list recipes[rec_num]["Ingredients"]
    ELSE IF read starts with "NAME:" THEN
      SET recipes[rec_num]["name"] = substring from index 5 to end
    ELSE IF read starts with "CATEGORY:" THEN
      SET recipes[rec_num]["Category"] = substring from index 9 to end
    ELSE IF read starts with "TIME:" THEN
      SET recipes[rec_num]["time"] = substring from index 5 to end
    ELSE IF read starts with "TAGS:" THEN
      SPLIT substring from index 5 by ","
      CONVERT into SET
      STORE in recipes[rec_num]["tags"]
    ELSE IF "|" exists in read THEN
      SPLIT read by "|"
      STORE parts as (name, quantity, unit)
      APPEND to recipes[rec_num]["Ingredients"]
    ELSE IF read starts with "===RECIPE===" THEN
      CONTINUE
    ELSE IF read starts with "===END===" THEN
      CONTINUE
    ENDIF
  ENDLOOP
  SET rec_count = number of recipes + 1
  CLOSE file
```

```
    DISPLAY "Loaded " + (rec_count - 1) + " recipes successfully"
END FUNCTION
```

10. Pseudocode for export_recipe

```
FUNCTION export_recipe
    loop forever
        INPUT rec_id
        CONVERT rec_id to uppercase
        REMOVE extra spaces from rec_id
        IF rec_id  in recipes THEN
            recipe_name = recipes[rec_id]["name"]
            file_name = "recipe_" + recipe_name + ".txt"
            OPEN file_name FOR WRITE
            WRITE "RECIPE: " + recipe_name
            WRITE "==============================="
            WRITE "Category: " + recipes[rec_id]["Category"]
            WRITE "Cooking Time: " + recipes[rec_id]["time"]
            WRITE "INGREDIENTS:"
            FOR each ingredient in recipes[rec_id]["Ingredients"]
                WRITE ingredient name + ": " + quantity + " " + unit
            ENDFOR
            convert tags set into comma separated string
            WRITE "TAGS: " + tags
            WRITE "==============================="
            CLOSE file
            DISPLAY "Recipe exported successfully"
            BREAK loop
        ELSE
            DISPLAY "Recipe ID not found! Try again"
        ENDIF
    END loop
END FUNCTION
```

11. Pseudocode for search_ingredient

```
FUNCTION search_ingredient
    loop forever
        TRY
            found = 0
            INPUT ask
            CONVERT ask to lowercase
            FOR each recipe_id in recipes
                FOR each item in recipes[recipe_id]["Ingredients"]
                    IF ask is in item name (lowercase) THEN
                        DISPLAY recipes[recipe_id]["name"]
```

```
                found=found+1
                BREAK loop(inner)
            ENDIF
        ENDFOR
    ENDFOR
    DISPLAY "We found " + found + " recipes related to " + ask
    BREAK loop
  CATCH
    DISPLAY "Enter valid ingredient name"
  ENDTRY
END loop
END FUNCTION
```

12. Pseudocode for cate_display

```
FUNCTION cate_display
  DISPLAY "01.BREAKFAST"
  DISPLAY "02.LUNCH"
  DISPLAY "03.DINNER"
  DISPLAY "04.DESSERT"
  DISPLAY "05.SNACK"
  DISPLAY "06.BEVERAGE"
  categories =["BREAKFAST", "LUNCH", "DINNER", "DESSERT", "SNACK", "BEVERAGE"]
  LOOP FOREVER
    TRY
      INPUT rec_cat AS integer
      global recipes
      IF rec_cat BETWEEN 1 AND 6
        selected_category = categories[rec_cat - 1]
        count = 0
        rec=[]
        FOR EACH key IN recipes
          IF recipes[key]["Category"] == selected_category
            count =count+1
            ADD recipes[key]["name"] TO rec
        DISPLAY "WE FOUND {count} RECIPES UNDER THIS CATEGORY"
        IF rec IS NOT EMPTY
          FOR EACH recipe_name IN rec
            DISPLAY recipe_name
        ELSE
          DISPLAY "Not found any recipe in that category"
        BREAK loop
      ELSE
        DISPLAY "Enter a valid category number"
    EXCEPT ERROR
      DISPLAY "Enter the number value of the category"
    ENDTRY
END FUNCTION
```

13. Pseudocode for filter_recipes_time

```
FUNCTION filter_recipes_time
   results =[]
   loop FOREVER
      INPUT time_choice
      IF time_choice LOWERCASE == "yes" THEN
         INPUT max_time as strinng
         INPUT min_time as string
         max_time_cal = INTEGER(max_time[0:2]) * 60 + INTEGER(max_time[3:5])
         min_time_cal = INTEGER(min_time[0:2]) * 60 + INTEGER(min_time[3:5])
         FOR EACH key IN recipes
            time_string = recipes[key]["time"]
            time_cal = INTEGER(time_string[0:2]) * 60 + INTEGER(time_string[3:5])
            IF min_time_cal < time_cal < max_time_cal THEN
               ADD key To results
            END IF
         END FOR
         BREAK loop
      ELSE IF time_choice LOWERCASE == "no" THEN
         DISPLAY "OK"
         BREAK LOOP
      ELSE
         DISPLAY "Enter a valid input"
      END IF
   END loop
END FUNCTION
```

14. Pseudocode for filter_recipes_no_ingre

```
FUNCTION filter_recipes_no_ingre
   global recipes
   global results
   loop FOREVER
      INPUT no_ingredient_choice
      IF no_ingredient_choice LOWERCASE == "yes" THEN
         INPUT max_ingre as integer
         INPUT min_ingre as integer
         FOR EACH key IN recipes
            ingre_count = length OF recipes[key]['Ingredients']
            IF min_ingre < ingre_count < max_ingre THEN
               IF key NOT IN results THEN
                  ADD key TO results
```

```
            END IF
          END IF
        END FOR
        BREAK loop
      ELSE IF no_ingredient_choice LOWERCASE == "no" THEN
        DISPLAY "OK"
        BREAK loop
      ELSE
        DISPLAY "Enter valid input"
      END IF
    END loop
END FUNCTION
```

15. Pseudocode for filter_recipes_category

```
FUNCTION filter_recipes_category
    global results
    loop FOREVER
      INPUT cate_choice
      IF cate_choice LOWERCASE == "yes" THEN
        INPUT categ_name
        FOR EACH key IN recipes
          IF UPPERCASE(categ_name) == recipes[key]["Category"] THEN
            IF key NOT IN results THEN
              ADD key TO results
            END IF
          END IF
        END FOR
        BREAK loop
      ELSE IF cate_choice LOWERCASE == "no" THEN
        DISPLAY "OK"
        BREAK loop
      ELSE
        DISPLAY "Enter valid input"
      END IF
    END loop
    DISPLAY "=====FILTER RESULT====="
    DISPLAY results
END FUNCTION
```

16. Pseudocode for change_name

```
FUNCTION change_name
    INPUT rec_id
    CONVERT rec_id TO UPPERCASE
    IF rec_id  IN recipes THEN
      CALL validate_recipe_name() AND STORE AS new_name
      recipes[rec_id]['name'] = new_name
```

```
      DISPLAY "Recipe {rec_id} updated successfully"
   ELSE
      DISPLAY "Entered Recipe ID is not available"
   END IF
END FUNCTION
```

17. Pseudocode for add_ingredient

```
FUNCTION add_ingredient
   INPUT rec_id
   CONVERT rec_id TO UPPERCASE
   IF rec_id  IN recipes THEN
      CALL validate_ingredient_name() AND STORE AS new_ingredient
      ADD new_ingredient TO recipes[rec_id]['Ingredients']
      DISPLAY "Recipe {rec_id} updated successfully"
   ELSE
      DISPLAY "Entered Recipe ID is not available"
   END IF
END FUNCTION
```

18. Pseudocode for del_ing_name

```
FUNCTION del_ing_name
   INPUT rec_id
   CONVERT rec_id TO UPPERCASE
   IF rec_id  IN recipes THEN
      INPUT del_ing_name
      CONVERT del_ing_name TO LOWERCASE
      found = FALSE
      FOR EACH item IN recipes[rec_id]["Ingredients"]
         IF del_ing_name == LOWERCASE(item[0]) THEN
            REMOVE item FROM recipes[rec_id]["Ingredients"]
            DISPLAY "Ingredient removed from Recipe {rec_id} successfully"
            found = TRUE
            BREAK loop
         END IF
      END FOR
      IF found == FALSE THEN
         DISPLAY "Ingredient not found"
      END IF
   ELSE
      DISPLAY "Entered Recipe ID is not available"
   END IF
END FUNCTION
```

19. Pseudocode for time_chan

```
FUNCTION time_chan
   INPUT rec_id
```

```
    CONVERT rec_id TO UPPERCASE
    IF rec_id IN recipes THEN
        CALL recipe_time_validation() AND STORE AS time
        recipes[rec_id]["time"] = time
        DISPLAY "Recipe {rec_id} time updated successfully"
    ELSE
        DISPLAY "Entered Recipe ID is not available"
    END IF
END FUNCTION
```

20. Pseudocode for cate_change

```
FUNCTION cate_change
    INPUT rec_id
    CONVERT rec_id TO UPPERCASE
    IF rec_id  IN recipes THEN
        CALL recipe_category_tags_validation() AND STORE AS rec_cate
        recipes[rec_id]["Category"] = rec_cate
        DISPLAY "Recipe {rec_id} Category updated successfully"
    ELSE
        DISPLAY "Entered Recipe ID is not available"
    END IF
END FUNCTION
```

21. Pseudocode for add_tags

```
FUNCTION add_tags
    INPUT rec_id
    CONVERT rec_id TO UPPERCASE
    IF rec_id  IN recipes THEN
        CALL recipe_tags() AND STORE AS tags
        recipes[rec_id]["tags"] = tags
        DISPLAY "Recipe {rec_id} tags updated successfully"
    ELSE
        DISPLAY "Entered Recipe ID is not available"
    END IF
END FUNCTION
```

22. Pseudocode for delete_recipe

```
FUNCTION delete_recipe
    INPUT rec_id
    CONVERT rec_id TO UPPERCASE
    IF rec_id  IN recipes THEN
        DELETE recipes[rec_id]
        DISPLAY "Recipe {rec_id} deleted successfully"
    ELSE
        DISPLAY "Entered Recipe ID is not available"
```

```
    END IF
END FUNCTION
```

23. Pseudocode for statis_category

```
FUNCTION statis_category
    bre_count = 0
    lun_count = 0
    din_count = 0
    des_count = 0
    sna_count = 0
    bev_count = 0
    DISPLAY "==================="
    DISPLAY "RECIPE BOOK SUMMARY"
    DISPLAY "==================="
    DISPLAY "Total Recipes: " + LENGTH OF recipes
    DISPLAY "Recipes by Category:"
    FOR EACH recipe_key IN recipes
        IF recipes[recipe_key]['Category'] == "BREAKFAST" THEN
            bre_count = bre_count + 1
        END IF
        IF recipes[recipe_key]['Category'] == "LUNCH" THEN
            lun_count = lun_count + 1
        END IF
        IF recipes[recipe_key]['Category'] == "DINNER" THEN
            din_count = din_count + 1
        END IF
        IF recipes[recipe_key]['Category'] == "DESSERT" THEN
            des_count = des_count + 1
        END IF
        IF recipes[recipe_key]['Category'] == "SNACK" THEN
            sna_count = sna_count + 1
        END IF
        IF recipes[recipe_key]['Category'] == "BEVERAGE" THEN
            bev_count = bev_count + 1
        END IF
    END FOR
    DISPLAY "BREAKFAST: " + bre_count
    DISPLAY "LUNCH: " + lun_count
    DISPLAY "DINNER: " + din_count
    DISPLAY "DESSERT: " + des_count
    DISPLAY "SNACK: " + sna_count
    DISPLAY "BEVERAGE: " + bev_count
END FUNCTION
```

24. Pseudocode for statis_time

```
FUNCTION statis_time
   DISPLAY "Cooking Time Distribution:"
   quick = 0
   medium = 0
   long = 0
   FOR EACH recipe_key IN recipes
      time_duration = recipes[recipe_key]['time']
      time_in_min = (INTEGER(time_duration[0:2]) * 60) + INTEGER(time_duration[3:5])
      IF time_in_min < 30 THEN
         quick = quick + 1
      ELSE IF 30 <= time_in_min < 60 THEN
         medium = medium + 1
      ELSE IF time_in_min >= 60 THEN
         long = long + 1
      END IF
   END FOR
   DISPLAY "01. Quick (<30 min): " + quick + " recipes"
   DISPLAY "02. Medium (30-60 min): " + medium + " recipes"
   DISPLAY "03. Long (>60 min): " + long + " recipes"
END FUNCTION
```

25. Pseudocode for statis_ingredient

```
FUNCTION statis_ingredient
   total_ing = 0
   max_ing_rec = 0
   max_rec_name = "not"
   min_ing_rec = 50
   min_rec_name = "not"
   FOR EACH recipe_key IN recipes
      len_ing_rec = LENGTH OF recipes[recipe_key]['Ingredients']
      total_ing = total_ing + len_ing_rec
      IF len_ing_rec > max_ing_rec THEN
         max_ing_rec = len_ing_rec
         max_rec_name = recipe_key
      END IF
      IF len_ing_rec < min_ing_rec THEN
         min_ing_rec = len_ing_rec
         min_rec_name = recipe_key
      END IF
   END FOR
```

```
    avg_rec = total_ing / LENGTH OF recipes
    DISPLAY "01. Average Ingredients per Recipe: " + avg_rec
    DISPLAY "02. Largest Recipe: " + max_rec_name + " with " + max_ing_rec + " ingredients"
    DISPLAY "03. Smallest Recipe: " + min_rec_name + " with " + min_ing_rec + " ingredients"
    DISPLAY "============================================="
END FUNCTION
```

26. Pseudocode for statis_ingredient

```
FUNCTION statis_ingredient
    total_ing = 0
    max_ing_rec = 0
    max_rec_name = "not"
    min_ing_rec = 50
    min_rec_name = "not"
    FOR EACH recipe_key IN recipes
        len_ing_rec = LENGTH OF recipes[recipe_key]['Ingredients']
        total_ing = total_ing + len_ing_rec
        IF len_ing_rec > max_ing_rec THEN
            max_ing_rec = len_ing_rec
            max_rec_name = recipe_key
        END IF
        IF len_ing_rec < min_ing_rec THEN
            min_ing_rec = len_ing_rec
            min_rec_name = recipe_key
        END IF
    END FOR
    avg_rec = total_ing / LENGTH OF recipes
    DISPLAY "01. Average Ingredients per Recipe: " + avg_rec
    DISPLAY "02. Largest Recipe: " + max_rec_name + " with " + max_ing_rec + " ingredients"
    DISPLAY "03. Smallest Recipe: " + min_rec_name + " with " + min_ing_rec + " ingredients"
    DISPLAY "============================================="
END FUNCTION
```

TEST CASE

TEST ID	FUNCTION	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	STATUS
001	validate_recipe_name()	RICE @ CURRY	Only letters, hyphen (-), apostrophe ('), and space are allowed.	Only letters,hyphen (-),apostrophe ('), and space are allowed.	PASS
002	validate_ingredient_details(ing_name)	kg	RETURN THE VALUE SUCCESSFULLY	==SYSTEM ERROR==	FAIL
003	validate_recipe_name()	Rice and curry	RETURN recipe_name	RETURN recipe_name	PASS
004	recipe_time_validation()	abcd	Format must be HH:MM	Format must be HH:MM	PASS
005	recipe_category_tags_validation()	lunch	RETURN THE VALUE SUCCESSFULLY	RETURN THE VALUE SUCCESSFULLY	PASS
006	save_rec()	-	RECIPES ARE SAVING SUCCESSFULLY	Error can't save your file	FAIL
007	search_ingredient()	12345	enter valid ingredient name	enter valid ingredient name	PASS
008	filter_recipes_time()	no	ok	ok	PASS
009	add_ingredient()	0001	Entered Recipe Id is not available	Entered Recipe Id is not available	PASS
010	del_ing_name()	RCP001	Ingredient removed from Recipe RCP001 successfully	==SYSTEM ERROR==	FAIL
011	filter_recipes_no_ingre()	Yes 5 3	SUCCESSFULLY ADDING TO THE LIST OF results	==SYSTEM ERROR==	FAIL
012	search_ingredient()	mango	we found 0 recipes related to mango ingredient name	we found 0 recipes related to mango ingredient name	PASS
013	validate_recipe_name()	QQ	Recipe name should contain between 3 and 50 characters.	Recipe name should contain between 3 and 50 characters.	PASS
014	statis_ingredient()	-	01.Average Ingredients per Recipe: 4	01.Average Ingredients per Recipe:4	PASS
015	validate_ingredient_details(ing_name)	00	Quantity must be greater than 0.	Quantity must be greater than 0.	PASS

TEST ID	REASON FOR FAIL
002	Not Denote the accepted quantity units as list (units = ['g', 'kg', 'ml', 'l', 'cup', 'tbsp', 'tsp', 'piece'])
006	Opened the file on read mode not in write mode
010	Did not mention the recipe as global
011	Did not mention the result list as global

REFLECTION

As a first-semester software design coursework, I was requested to design a command-line recipe management system, which performs basic tasks such as adding recipes, viewing the recipes by their ID, and creating basic statistical reports.

Initially, the coursework requirements were not fully reviewed , just tried to do it task by task by creating relevant pseudo code and coding. While finishing task 2, I realized I was coding out of the requirement, and then I read the coursework requirement and made the required modification to the code

During the development process, AI-based tools such as ChatGPT and Gemini were utilized to assist in debugging and understanding errors that could not be resolved independently. Comments near the code where I took the support of the AI tool as a credential. Additionally, this process contributed to learning new concepts that were not taught during lectures or tutorials.

For me personally, file handling was a little bit confusing in Python, but this coursework gave me confidence in file handling and nested dictionary parts. I have come across more than a hundred errors when executing the programs, such as indentation errors, missing variable assignments, calling variable names, and so on, from which a few of the main test cases I have added to the above test case table, along with a table to denote the reason for failure.

Given additional time, further enhancements could have been implemented, such as sorting recipes alphabetically and adding more advanced functionalities. This coursework also helped me understand the importance of planning and requirement analysis in software development.

Overall, this assignment was a valuable learning experience that strengthened my technical skills and improved my approach to problem-solving.